



Universidad
de Jaén

| Fundamentos de Arquitectura de Computadores

Práctica 1

Introducción al lenguaje ensamblador

El banco de registros · Aritmética · Datos en memoria · Direccionamiento

Grado en Ingeniería Informática · Universidad de Jaén

Objetivos de la práctica

Primera práctica evaluable: tus primeros programas en ensamblador RISC-V, de los registros a la memoria.

- ◆ Manejar el **banco de registros** y sus alias de la ABI.
- ◆ Operar con las instrucciones **aritméticas**: sumas, restas, productos, divisiones y restos.
- ◆ Declarar **datos en memoria** y moverlos con `lw`, `sw` y `la`.
- ◆ Mostrar resultados por consola con `ecall`.
- ◆ Entender los cuatro **modos de direccionamiento** de RISC-V.
- ◆ Tomar **decisiones** con los saltos condicionales: el *if* del ensamblador.

Estas diapositivas son el guion de trabajo de las sesiones. El detalle completo, con todos los ejemplos explicados, está en el **capítulo 1 del documento de consulta** de la asignatura.

El banco de registros

Registro	Alias	Uso
x0	zero	Siempre vale 0
x1	ra	Dirección de retorno
x2	sp	Puntero de pila
x5-x7, x28-x31	t0-t6	Valores temporales
x8-x9, x18-x27	s0-s11	Registros preservados
x10-x17	a0-a7	Parámetros y valores de retorno

Lo esencial

- ◆ **32 registros** de propósito general, de **32 bits** cada uno.
- ◆ Cada registro tiene un **alias** con significado, definido por la ABI.
- ◆ Usa siempre los alias: **a0** dice más que **x10**.
- ◆ **zero** es de solo lectura: escribir en él no tiene efecto.

Instrucciones aritméticas

El repertorio

- ◆ `add` / `addi` — suma (con registro o con inmediato).
- ◆ `sub` — resta.
- ◆ `mul` — multiplicación.
- ◆ `div` — cociente de la división entera.
- ◆ `rem` — resto de la división entera.

El primer operando es siempre el **destino**; solo `addi` admite un inmediato.

```
# t0 = 23, t1 = 4
li    t0, 23
li    t1, 4

add   a0, t0, t1    # a0 = 27
sub   a1, t0, t1    # a1 = 19
mul   a2, t0, t1    # a2 = 92
div   a3, t0, t1    # a3 = 5
rem   a4, t0, t1    # a4 = 3
```

Instrucciones y pseudoinstrucciones

Instrucción

Existe de verdad en la máquina: tiene su propio formato y su codificación binaria. Una línea de código fuente, una palabra de código máquina.

Seudoinstrucción

No existe en la máquina. Es una abreviatura cómoda que el **ensamblador** traduce por ti a una o varias instrucciones reales.

En el **panel derecho del editor** de Ripes ves en qué se convierte cada línea: ahí se nota cuáles eran pseudoinstrucciones y cuántas instrucciones reales genera cada una.

lo que escribes -> lo que ejecuta la maquina

```
li    t0, 5           ->  addi t0, zero, 5
mv    a0, t0          ->  addi a0, t0, 0
j     fin             ->  jal  zero, fin
beqz  t0, fin          ->  beq  t0, zero, fin
nop                    ->  addi zero, zero, 0

la    t1, dato         ->  auipc t1, ...
                        ->  addi t1, t1, ...
```

Hablar con la consola: ecall

Las instrucciones solo saben de registros y memoria. Para todo lo demás se lo pedimos al entorno con **ecall**, siguiendo una convención fija:

- ◆ **a7** lleva el **código del servicio**.
- ◆ **a0** lleva el **argumento** (y recoge el resultado, si lo hay).

La lista completa de servicios está en **Help** → **System calls** dentro de Ripes.

Los que usaremos

a7	Servicio	a0
1	Imprimir entero	el entero
4	Imprimir cadena	su dirección
34	Imprimir en hexadecimal	el entero
35	Imprimir en binario	el entero
10	Terminar el programa	—

Ejemplo guiado 1 · Sumar y mostrar

```
# a1, a2: sumandos
# a0:      resultado
# a7:      servicio de ecall

addi a1, zero, 3    # a1 = 3
addi a2, zero, 7    # a2 = 7
add  a0, a1, a2      # a0 = a1 + a2

addi a7, zero, 1
ecall                # imprime a0
addi a7, zero, 10
ecall                # fin
```

Qué observar al ejecutarlo

- ◆ Escríbelo en el **Editor** y ejecuta con **F8**.
- ◆ En la lista de registros, **a1**, **a2** y **a0** cambian y se resaltan.
- ◆ La **consola** muestra el 10.
- ◆ En el panel derecho, cada línea tiene su **código máquina**.

No existe una instrucción para cargar un inmediato: se suma sobre **zero**. La seudoinstrucción **li a1, 3** hace exactamente eso, y **mv** copia un registro en otro.

Datos en memoria

Declarar

- ◆ `.data` — datos con valor inicial.
- ◆ `.bss` — datos sin inicializar.
- ◆ `.text` — el código.
- ◆ `.word` — reserva un entero de 32 bits (4 bytes).

Leer y escribir

- ◆ `lw rd, etiqueta` — carga el dato.
- ◆ `la rd, etiqueta` — carga la dirección.
- ◆ `lw rd, desp(rbase)` — carga con desplazamiento.
- ◆ `sw ro, desp(rbase)` — escribe en memoria.

Las direcciones

- ◆ `.data` empieza en `0x10000000`.
- ◆ Palabras consecutivas se separan **4 bytes**.
- ◆ El elemento i vive en $\text{base} + 4 \cdot i$.

Ejemplo guiado 2 · Leer, sumar y guardar

```

.data
n1: .word 7
n2: .word 3

.bss
res: .word 0

.text
lw  a1, n1           # a1 = 7
lw  a2, n2           # a2 = 3
add a0, a1, a2       # a0 = 10
la  t0, res          # t0 -> res
sw  a0, 0(t0)        # (res) = 10

li  a7, 10
ecall

```

0x1000028	X	X	X	X	X
0x1000024	X	X	X	X	X
0x1000020	X	X	X	X	X
0x100001c	X	X	X	X	X
0x1000018	X	X	X	X	X
0x1000014	X	X	X	X	X
0x1000010	X	X	X	X	X
0x100000c	X	X	X	X	X
0x1000008	0	0	0	0	0
0x1000004	3	3	0	0	0
0x1000000	7	7	0	0	0
0x0fffffc	X	X	X	X	X
0x0fffff8	X	X	X	X	X
0x0fffff4	X	X	X	X	X
0x0fffff0	X	X	X	X	X

Tras ejecutar, abre **Memory**, salta a `.data` con *Go to section* y pon *Display type* en *Signed*: ahí están el 7 y el 3. El 10 espera en `.bss` (`0x11000000`).

Los bytes de una palabra

Una palabra ocupa **4 bytes**, y RISC-V los guarda en orden **little-endian**: el byte de **menor peso** ocupa la dirección **más baja**.

```
val: .word 0x1FAE8A2A
```

2A

val + 0

8A

val + 1

AE

val + 2

1F

val + 3

El **0x1F**, que es el byte de mayor peso, aparece el último.

Byte a byte

Además de las que mueven **palabras enteras**, el repertorio tiene instrucciones que trabajan **a nivel de byte**: unas cargan un solo byte en un registro, con o sin extensión de signo, y otra escribe únicamente el byte de menor peso. Las tienes en el **Anexo A** del manual.

Compruébalo en la vista **Memory**, donde cada palabra aparece desglosada en sus cuatro bytes.

Un dato nuevo: el texto

- ◆ Cada carácter es un número: su código **ASCII** (1 byte).
- ◆ `.string` guarda los bytes y añade un **0 final** que marca el fin.
- ◆ El servicio **4** imprime desde la dirección de `a0` hasta ese 0.

Carácter	H	o	l	a	\n	fin
Byte	48	6F	6C	61	0A	00

```
.data
hola: .string "Hola, RISC-V\n"

.text
la a0, hola    # direccion de la cadena
li a7, 4       # servicio 4: imprimir
ecall

li a7, 10
ecall
```

Números negativos: el complemento a dos

- ◆ Un registro son 32 bits sin ninguna marca de signo: es el **convenio** el que decide cómo se leen.
- ◆ Si el **bit más alto** vale 1, el número se interpreta como **negativo**.
- ◆ Para obtener el negativo de un valor se **invierten todos sus bits** y se le **suma 1**.
- ◆ Con 32 bits el rango va de **-2147483648** a **2147483647**.

Un mismo patrón de bits admite varias lecturas: cambia **Display type** entre *Signed* y *Hex* en las vistas de registros y de memoria.

```
# de 163 a -163
```

```
163      0x000000A3
invertir  0xFFFFFFFF
sumar 1   0xFFFFFFFF <- -163
```

```
# comprobarlo en Ripes
```

```
li    t0, -163
mv    a0, t0
li    a7, 34      # imprimir en hexadecimal
ecall                      # 0xffffffff5d
mv    a0, t0
li    a7, 35      # imprimir en binario
ecall                      # 0b1111...01011101
```

¿De dónde salen los operandos? (1 de 2)

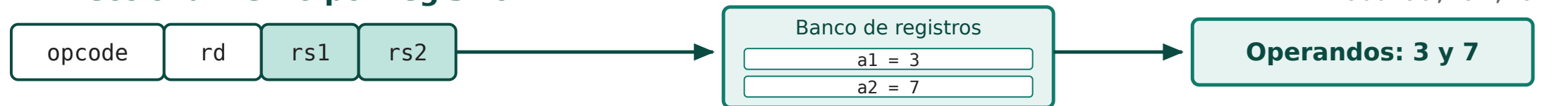
En los dos primeros modos el operando está a mano: viaja dentro de la propia instrucción o vive en un registro.

1. Direcccionamiento inmediato



El operando (el 5) es una constante que viaja dentro de la propia instrucción: no hay que buscarlo en ningún otro sitio.

2. Direcccionamiento por registro



Los campos rs1 y rs2 indican qué registros del banco contienen los operandos (aquí, a1 y a2).

Inmediato

```
addi a0, a1, 5
```

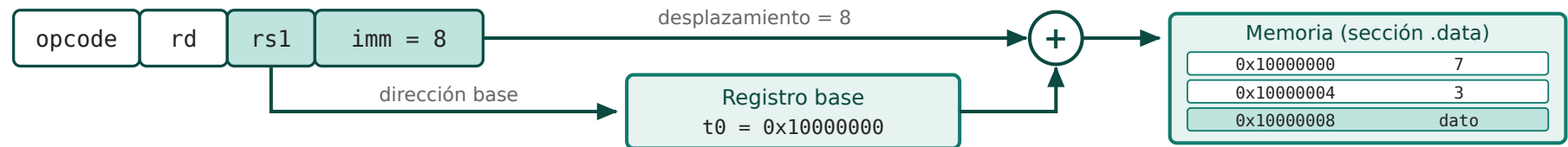
Por registro

```
add a0, a1, a2
```

¿De dónde salen los operandos? (2 de 2)

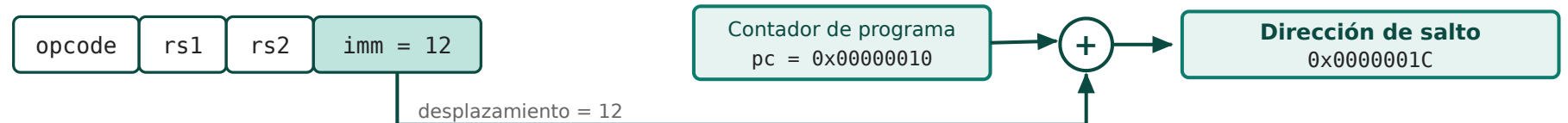
En los otros dos, la instrucción no lleva el dato: lleva la receta para calcular la dirección donde está.

3. Direccionamiento indexado (base + desplazamiento)



La dirección del dato se calcula sumando a la dirección base (contenida en rs1 = t0) el desplazamiento imm: $0x10000000 + 8 = 0x10000008$; de esa posición de memoria (fila sombreada) se lee el dato. Es el modo de cargas y almacenamientos.

4. Direccionamiento relativo al pc



Si la condición se cumple, el pc pasa a valer $pc + \text{desplazamiento}$: $0x00000010 + 12 = 0x0000001C$, y la ejecución continúa por la instrucción almacenada en esa dirección de la sección .text.

Indexado

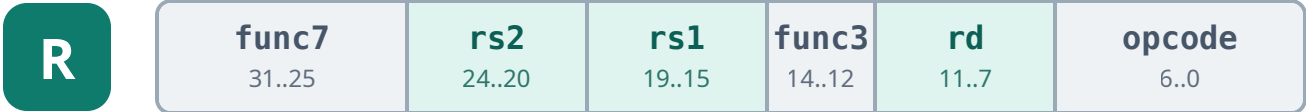
`lw a0, 8(t0)`

Relativo al pc

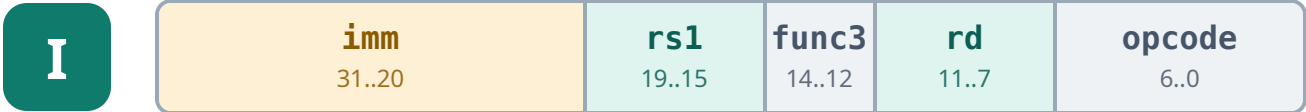
`beq a1, a2, etiqueta`

Tipos de instrucción: cómo se reparten los 32 bits

Todas las instrucciones ocupan 32 bits; lo que cambia de un tipo a otro es cómo se reparten entre la operación, los registros y el inmediato.



Tres registros: dos operandos y el destino.
`add a0, a1, a2`



Dos registros y un inmediato de 12 bits con signo.
`addi a0, a1, 5` `lw a0, 8(t0)`



Dos registros de origen; el inmediato va en dos trozos.
`sw a0, 4(t0)` y los saltos, como `beq`



El registro de destino y un inmediato de 20 bits.
`lui t0, 0x10000` `auipc`

registro inmediato operación (opcode, func3 y func7)

Los registros ocupan siempre las mismas posiciones, y el `opcode`, los 7 bits de menor peso. La codificación campo a campo, con ejemplos reales, está en el apartado 1.5 del manual.

El truco de la dirección de 32 bits

- ◆ En una instrucción de 32 bits **no cabe** una dirección completa.
- ◆ **lui** coloca 20 bits **desplazados 12 a la izquierda**, dejando ceros abajo.
- ◆ **addi** suma los 12 bits bajos: sin acarreo posible, las piezas **encajan**.
- ◆ **la** hace esto por ti (en Ripes, con **auipc** + **addi**).

Compruébalo: escribe **la t0, dato** y mira en el panel derecho las dos instrucciones reales que genera.

```
# dato esta en 0x10000123

lui  t0, 0x10000
# t0 = 0x10000000 (12 ceros abajo)

addi t0, t0, 0x123
# t0 = 0x10000123: encajado

# equivalente: la t0, dato
```

Tomar decisiones: saltos condicionales

- ◆ El `pc` guarda la dirección de la **siguiente instrucción**; un salto lo modifica y la ejecución continúa en una **etiqueta**.
- ◆ `beq` / `bne` : salta si iguales / distintos.
- ◆ `bge` / `blt` : salta si mayor o igual / menor (`bgeu` y `bltu` para valores sin signo).
- ◆ Seudoinstrucciones útiles: `bgt` , `ble` , `beqz` , `bnez` ...
- ◆ `j etiqueta` salta **siempre**: al terminar el bloque del *else*, sirve para saltarse el del *if*.

Esríbelas en Ripes y mira en el panel derecho el código real que genera cada pseudoinstrucción.

```
# esquema de un if-else

    bge a2, a1, entonces # si se cumple, salta
    ...                 # bloque del no (else)
    j fin               # saltarse el del si
entonces:
    ...                 # bloque del si (if)
fin:
    ...                 # continua el programa
```

Ejemplo guiado 3 · Un condicional

- ◆ Calcula la **diferencia positiva** de dos valores.
- ◆ Si $a2 \geq a1$ el salto lleva a **mayora2**; si no, se resta y **j** **fin** evita el otro camino.
- ◆ Equivale al C: `if (a2 >= a1) a0 = a2 - a1; else a0 = a1 - a2;`
- ◆ Ambos caminos se **reencuentran** en **fin**.

Ejecuta paso a paso y observa el **pc** en cada salto. Cambia los valores iniciales para recorrer el otro camino.

```
# a1, a2: valores a comparar
# a0: diferencia positiva
# a7: servicio de ecall
.text
li a1, 12
li a2, 7

bge a2, a1, mayora2
sub a0, a1, a2
j fin
mayora2:
sub a0, a2, a1
fin:
li a7, 1
ecall
li a7, 10
ecall
```

Cómo trabajaremos

- ◆ En cada sesión resolverás los **ejercicios del guion** de la práctica en Ripes.
- ◆ Cada ejercicio se guarda en su propio fichero **.s** (menú *File*).
- ◆ Comprueba siempre el resultado: **consola**, **registros** y vista **Memory**.
- ◆ La entrega se realiza en **Platea**, en la tarea de la práctica; consulta allí el plazo y el formato.

Ten a mano el documento de consulta (capítulo 1) y el anexo con el repertorio de instrucciones: están pensados para usarse mientras programas.

PRÁCTICA 1 EN MARCHA

¿Qué viene después?

- 1 **Lee el guion** de la Práctica 1: ahí están los tres ejercicios y cómo se entregan.
- 2 **Resuelve los ejercicios** con tu pareja: tienes **tres semanas**.
- 3 **Entrega** en Platea el `.zip` con los tres ejercicios antes de la fecha límite.
- 4 **Defiende** la práctica en la clase siguiente a la entrega, explicando tu código.

Recursos: documento de consulta de la asignatura (capítulo 1 y Anexo A) · repositorio github.com/mortbopet/Ripes